DATA SCIENCE CERTIFICATION PROJECT

AI Book Recommendation System

Personalized Reading, Powered by Collaborative Filtering

Tuned SVD (Surprise) · Streamlit Deployment

BOOK-CROSSING DATASET – P681 (GROUP 6)

THE PEOPLE BEHIND IT

Mentor & Team Members

MENTOR

PRAJWAL A N

S. S. G. LIKHITHA

K. PAVITHRA

E. GOWRAV CHOWDARY

A. LEELA SAINATH GOUD

R. SIRISHA

WHY THIS PROJECT

Objective

Personalize Discovery

Predict which unread books a specific user would rate highly, based on their past reading signal.

Model Latent Taste

Use collaborative filtering (SVD matrix factorization) to learn hidden preference patterns from the rating matrix.

Handle Extreme Sparsity

Work reliably across a matrix that is over 99.9% empty — most users rate only a handful of books.

Deploy as a Web App

Serve real-time, ranked recommendations through an interactive Streamlit dashboard.

Enrich Discovery Beyond Ratings

Layer trending, mood-based and surprise discovery modes on top of the core predictions.

Dataset Details

Source: Books.csv · Ratings.csv · Users.csv (Book-Crossing Dataset) | Binary/Explicit-Implicit Rating Task

266,726
Books Catalogued

277,610
Registered Users

25,000
Ratings (sample)

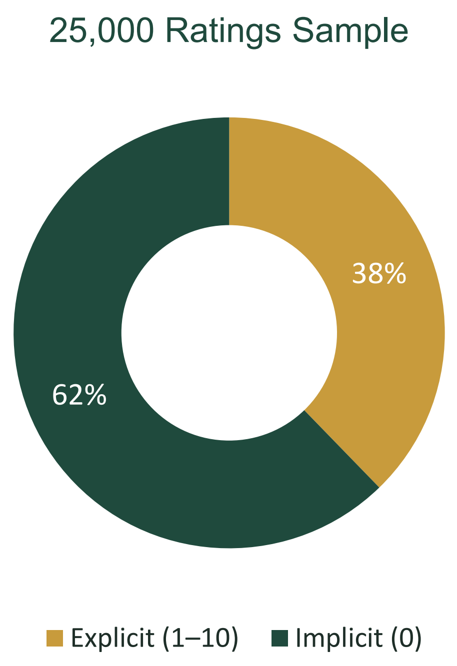
100,666
Unique Authors

99.99%
Matrix Sparsity

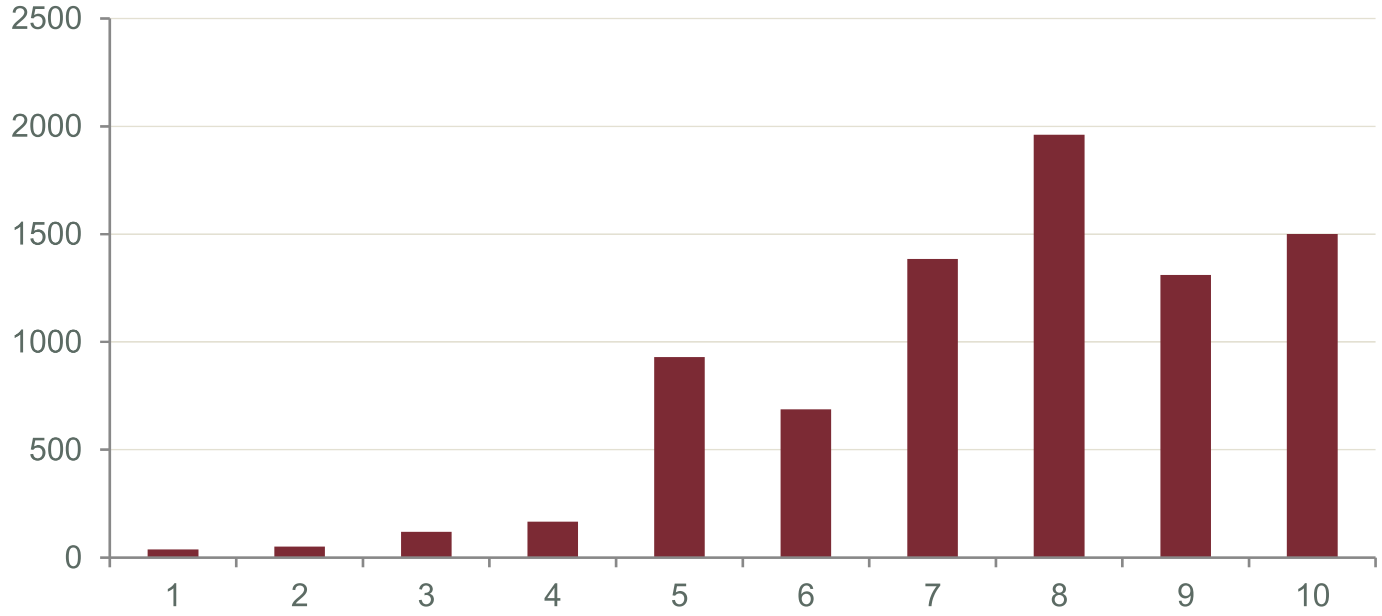
Key Fields at a Glance

Field	Source	Description
Book-Rating	Ratings.csv	0 = implicit interaction · 1–10 = explicit star rating
ISBN / Book-Title	Books.csv	Book identity, author, publisher, cover art URLs
User-ID / Age / Location	Users.csv	Reader demographics used for segmentation
Type	Engineered	Flags each row as Explicit or Implicit signal

Ratings — Explicit vs Implicit Signal



Explicit Rating Distribution (1–10 scale)



Positive Skew

Explicit ratings lean heavily toward 7–10 — readers mostly rate books they liked.

Implicit Majority

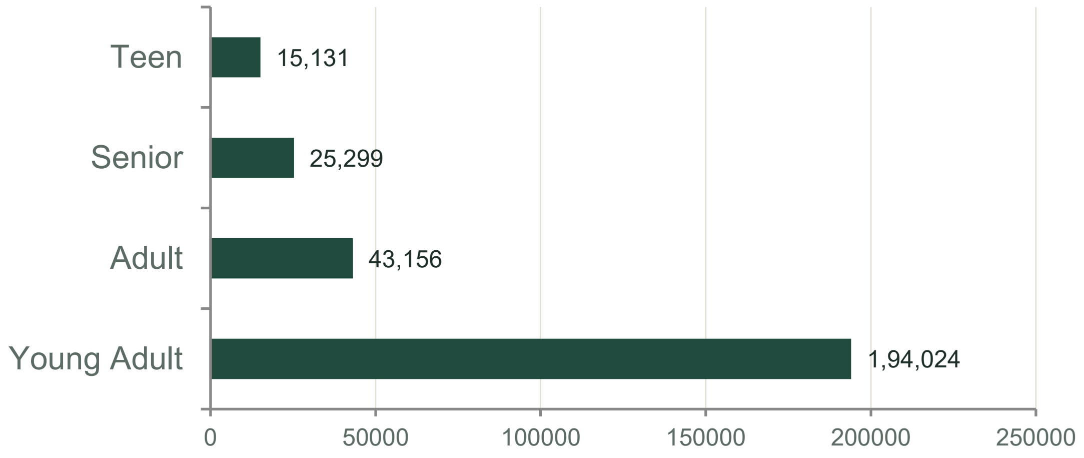
62% of interactions carry no star rating (Type = Implicit), a common Book-Crossing pattern.

Average Explicit Rating

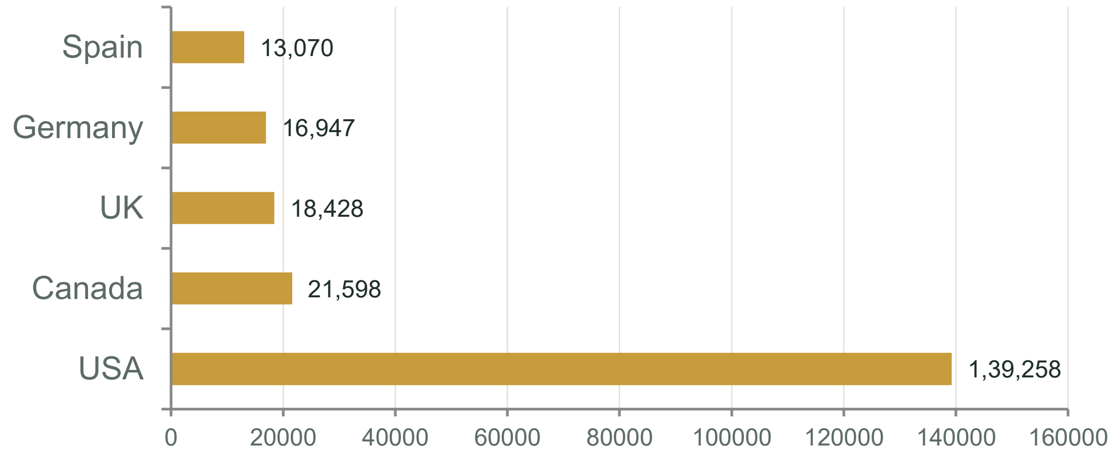
≈ 7.6 / 10 across all rated books in the sample.

Who Are the Readers?

Users by Age Group



Top Reader Countries

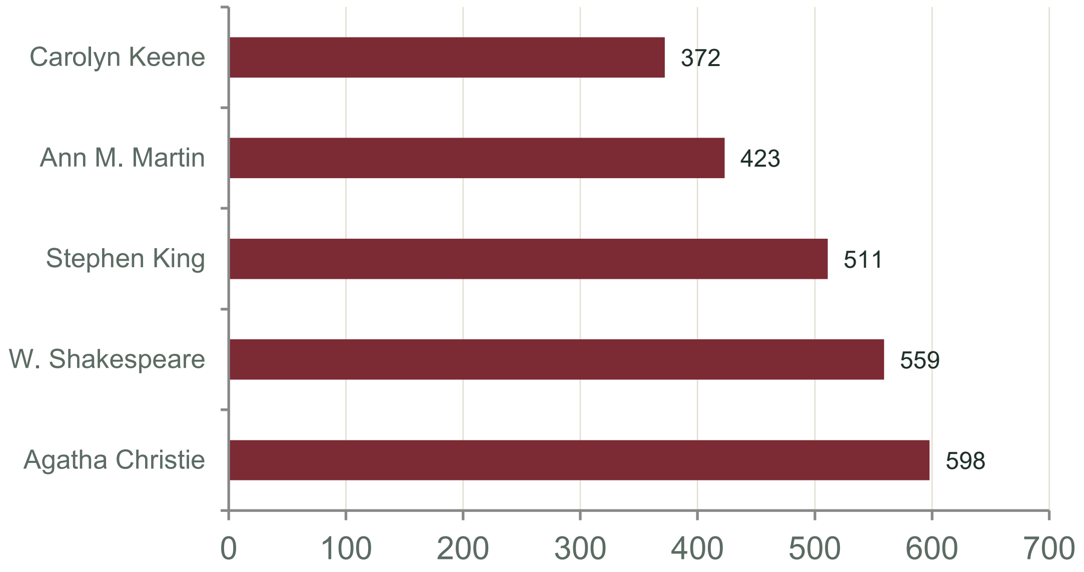


Why Sparsity Matters

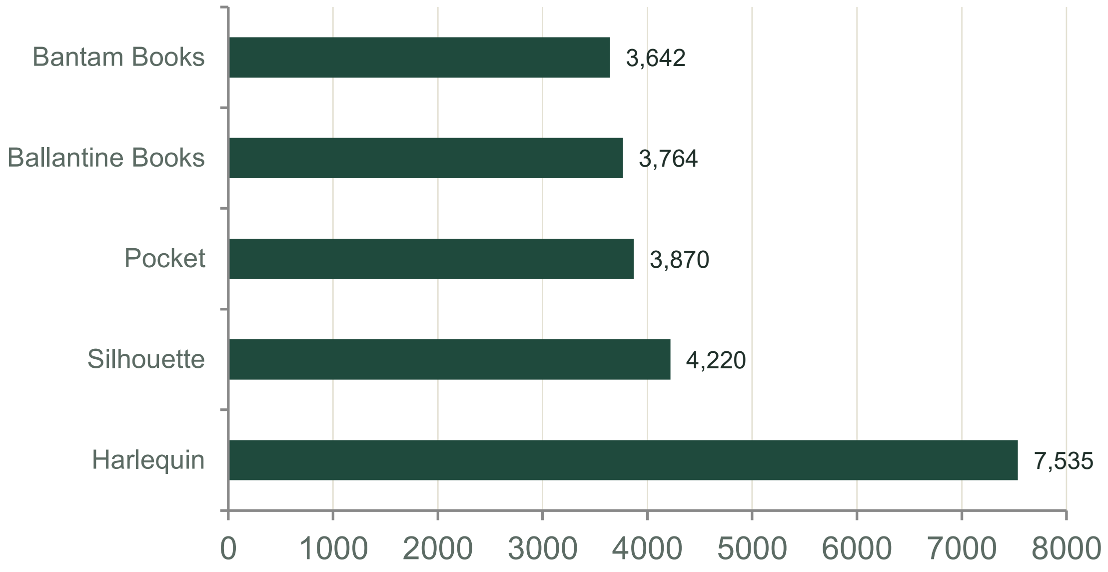
The rated user × book matrix spans 10,254 users × 20,032 books — over 205 million possible entries — yet only 25,000 are filled. That is 99.99% sparsity. Neighbourhood methods (like plain KNN) struggle to find enough overlap between users at this scale, which is exactly the gap matrix factorization (SVD) is built to close: it learns dense latent factors instead of relying on direct user-user overlap.

Most Prolific Authors & Publishers

Authors With the Most Titles



Publishers With the Most Titles



Long-tail catalogue: 100,666 distinct authors and 16,396 publishers sit behind 266,726 books — most titles have very few ratings, reinforcing the cold-start and sparsity challenge tackled in modelling.

GETTING MODEL-READY

Data Preprocessing & Feature Engineering

01

Merge the Three Sources

Books, Ratings and Users joined on ISBN / User-ID into a single modelling table (final_df, 21,901 rows).

02

Split Explicit vs Implicit

The Type column separates 0-value browsing signal (Implicit) from genuine 1–10 star ratings (Explicit) used for training.

03

Engineer New Features

Author_Popularity, Book_Age, Title_Length, Average_Rating and Rating_Count derived to enrich each book profile.

04

Encode Users & Books

User-ID and ISBN mapped to dense integer indices (user_idx, book_idx) for the factorization model.

05

Bin Reader Age Groups

Raw Age converted into Teen / Young Adult / Adult / Senior segments for downstream analysis.

06

Train-Test Split

Explicit ratings split 80/20 with Surprise's train_test_split for unbiased RMSE / MAE evaluation.

Six Classical Recommendation Approaches

NON-PERSONALIZED

N/A

Popularity-Based

Groups books by title, ranks by average rating & rating count. Same list for every user — but works instantly for brand-new readers with zero history.

CONTENT FILTERING

N/A

Content-Based (TF-IDF)

Combines Title + Author + Publisher into text, then measures cosine similarity across 269 popular books — finds books alike in subject, not user taste.

COLLABORATIVE

1.862

Baseline Only

Global average rating adjusted by per-user and per-book bias — no factorization at all. Surprisingly strong: beats untuned SVD.

→ MOVES TO TUNING

COLLABORATIVE

1.864

SVD

Matrix factorization — decomposes the sparse rating matrix into latent user & book factors. Nearly tied with Baseline before tuning.

→ MOVES TO TUNING

COLLABORATIVE

1.912

KNN Basic (k=20)

Finds the 20 most similar users/books via a similarity matrix. Sparsity limits how many true neighbours actually exist.

COLLABORATIVE

2.088

NMF

Non-negative matrix factorization (20 factors, 20 epochs) — weakest of the four collaborative filtering models tested.

RMSE shown where applicable (lower is better) · evaluated on the explicit-rating subset (9,436 ratings)

Three Deep Learning Approaches

DEEP LEARNING

4.251

NCF (Neural CF)

50-dimensional embeddings for users & books, concatenated through dense layers. Trained just 10 epochs on ~9.4K explicit ratings — validation loss climbed while training loss crashed, a clear overfitting signal.

DEEP LEARNING

4.258

DeepFM

Adds a factorization-machine (Multiply) component alongside a deep branch. Same overfitting pattern as NCF — the architecture needs orders of magnitude more training data to pay off.

DEEP LEARNING

4.143

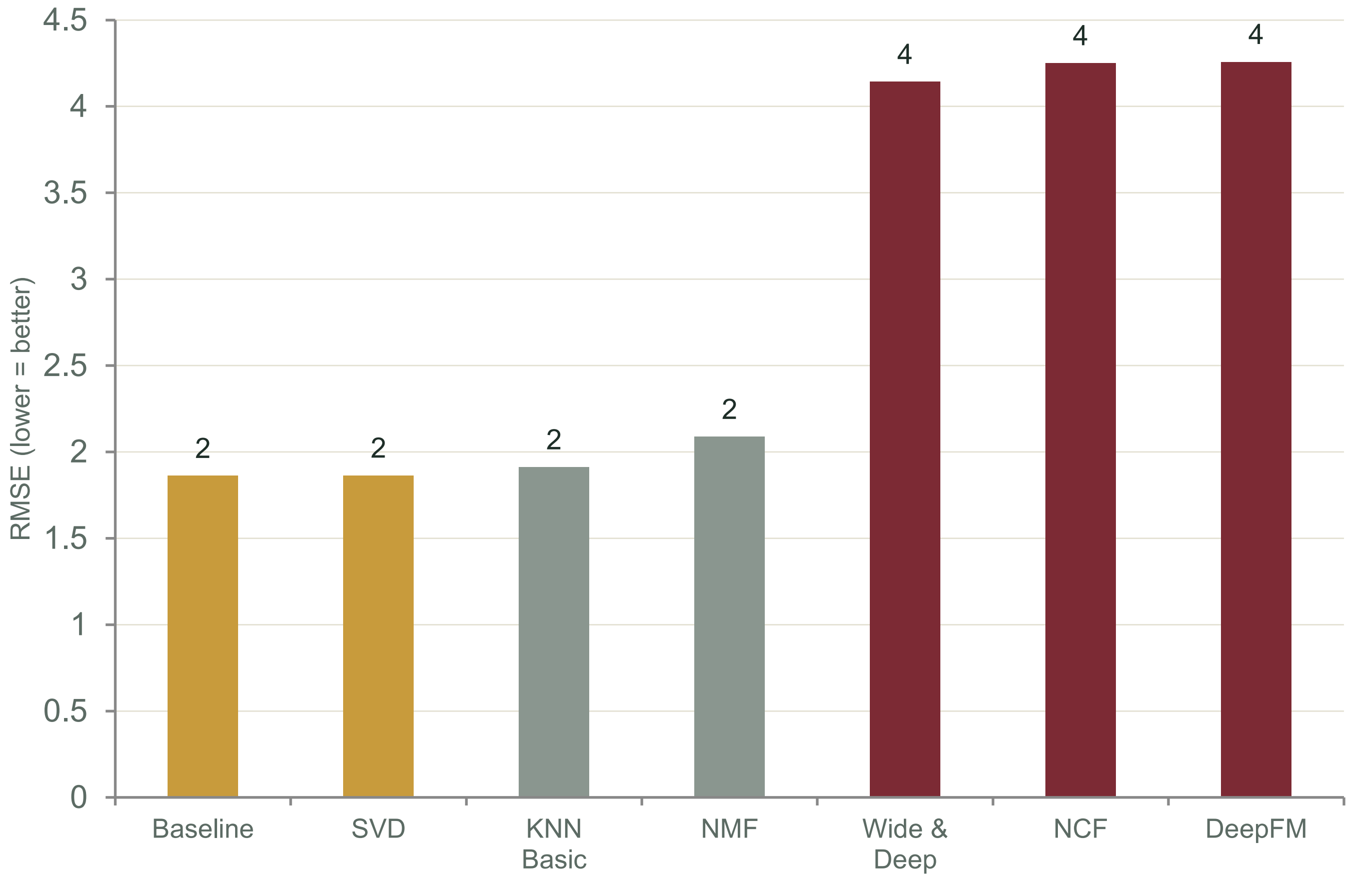
Wide & Deep

Separate wide (memorisation) and deep (generalisation) branches trained jointly. Best of the three neural nets, but still more than double the error of the simplest classical model.

Why Deep Learning Underperformed Here

With only ~9,436 explicit ratings, these architectures don't have enough signal to learn meaningful embeddings — they memorise noise instead of genuine patterns. Industry-scale neural recommenders typically train on hundreds of thousands to millions of interactions. This is a valid, expected result: it confirms that classical collaborative filtering is the right choice at this data scale, not a shortcoming in the implementation.

Model Comparison — RMSE Benchmark



Baseline & SVD Lead: ~1.86

Nearly tied — both carried forward into GridSearchCV tuning as finalists.

KNN / NMF Trail Slightly

1.91 and 2.09 — sparsity limits neighbourhood and factorization quality further.

Deep Learning Underperforms

NCF, DeepFM and Wide & Deep score 4.1–4.3 — over double the error. With only ~9.4K explicit ratings, these architectures overfit fast; they typically need far more data to beat classical CF.

Popularity & Content-Based

Not RMSE-comparable — they rank books rather than predict individual scores, but remain useful for cold-start users.

FINAL MODEL

Tuned SVD — Matrix Factorization

How SVD Works

1 **Factorize the Rating Matrix**

The sparse user × book matrix is decomposed into dense user-factor and item-factor vectors.

2 **Learn 50 Latent Dimensions**

n_factors = 50 — each user and book gets 50 learned “taste” dimensions.

3 **Add Bias Terms**

Global mean + per-user bias + per-book bias capture individual rating tendencies.

4 **Predict via Dot Product**

predicted rating = bias terms + (user vector · book vector), for every unrated book.

Hyperparameter Tuning

n_factors = 50

Latent dimensions per user & book — enough to capture nuance without overfitting the small explicit set.

n_epochs = 50

Stochastic gradient descent passes over the training ratings.

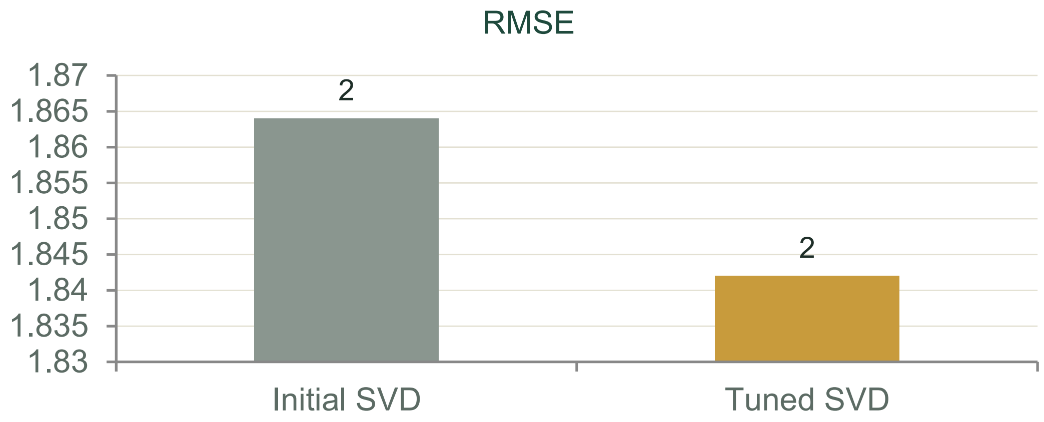
lr_all = 0.005

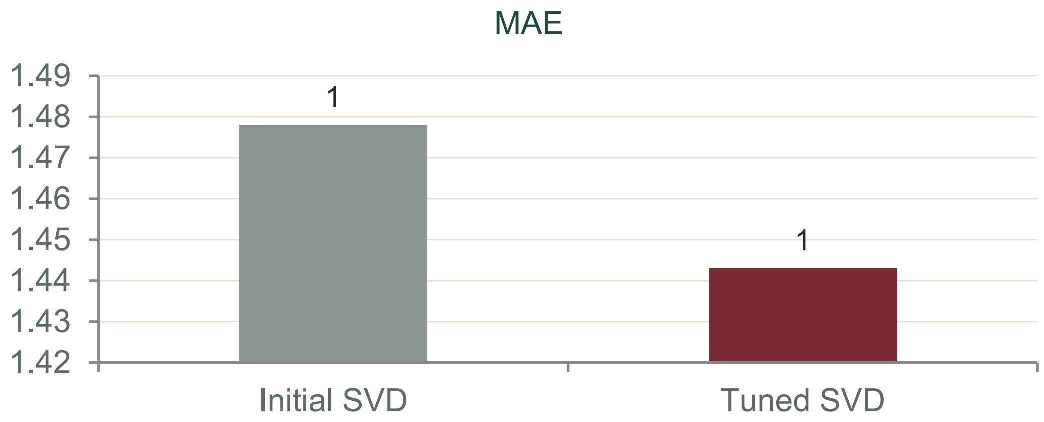
Learning rate applied uniformly to biases and factor vectors.

reg_all = 0.05

L2 regularisation strength controlling overfitting on sparse users/books.

Initial vs Tuned SVD — GridSearchCV Results





Key Findings from Tuning

- **Real, Measurable Improvement**

  GridSearchCV lifts SVD from RMSE 1.864 → 1.842 and MAE 1.478 → 1.443 — a genuine gain, not noise.

- **Stronger Regularisation Wins**

  reg_all rises from 0.02 to 0.05 and n_epochs from 10 to 50 — more training passes, better controlled.

- **Beats Its Closest Rival**

  Tuned SVD (1.842) narrowly edges out Tuned Baseline (1.848) — the two finalists from the 9-model bake-off.

- **3-Fold Cross-Validated**

  GridSearchCV searched 81 parameter combinations with 3-fold CV before selecting the winning configuration.

MODEL EVALUATION

Final Model — Performance Summary

1.84

RMSE (1–10 scale)

Root mean squared error on held-out explicit ratings

1.44

MAE (1–10 scale)

Average absolute prediction error

50

Latent Factors

Learned taste dimensions per user & book

9,436

Explicit Ratings Used

Training + evaluation sample after filtering

Complete ML Pipeline

Raw Data (266K books) → Merge & Clean → Feature Engineering → 9 Models Benchmarked → GridSearchCV Tuning → Tuned SVD → RMSE 1.84

Nine approaches were benchmarked — from simple popularity ranking to deep neural architectures. Tuned SVD emerged as the overall winner, narrowly ahead of Tuned Baseline. On a 1–10 rating scale, predictions land within roughly 1.4 stars of the true rating on average — strong enough to rank unseen books meaningfully for each reader despite extreme data sparsity.

How Recommendations Work

Select User ID → Exclude Already-Rated Books → Predict Rating for Every Candidate → Rank by Predicted Score → Return Top-N with Cover & Metadata

Inside recommend_books()

- rated_books = ratings already rated by the selected User-ID are removed from the candidate pool.
- model.predict(user_id, isbn) scores every remaining candidate book with the tuned SVD model.
- Predictions are sorted descending and the Top-N (5–20, adjustable) are merged back with Book-Title, Author, Publisher and cover art.

BUILT WITH

Technology Stack

Python 3

Core language for the entire pipeline — from EDA to model training and deployment.

Pandas & NumPy

Data loading, merging, cleaning and feature engineering across all three source files.

Matplotlib & Seaborn

Visualisation for ratings distributions, sparsity and author/publisher analysis.

Scikit-Surprise

SVD, KNN, NMF and Baseline model training, cross-validation and RMSE/MAE evaluation.

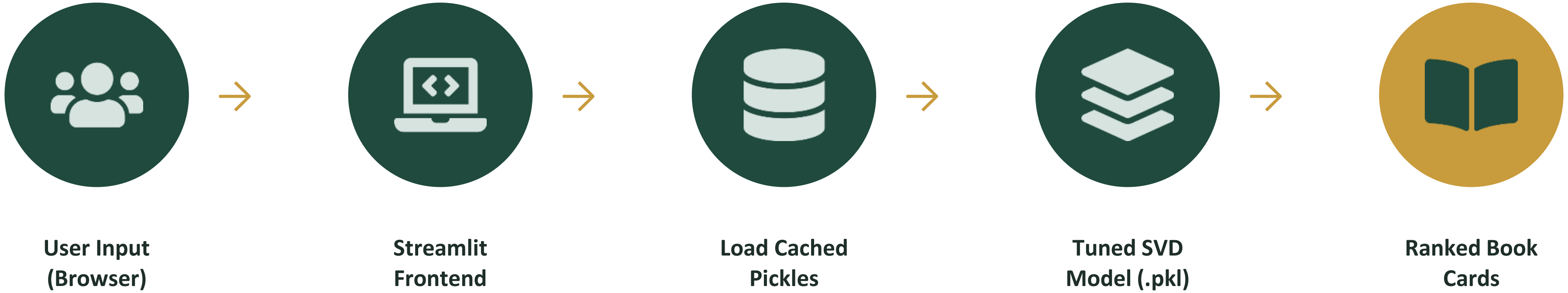
Streamlit

Converts the trained model into an interactive web app with sidebar controls and live predictions.

Pickle

Serialises books, ratings, users and the tuned SVD model so the app loads instantly without retraining.

Deployment Architecture — Streamlit App



App Features

User ID Selector

Sidebar dropdown lets any reader in the dataset be scored instantly.

Adjustable Top-N

Slider controls how many recommendations to return (5–20).

Live Dashboard Metrics

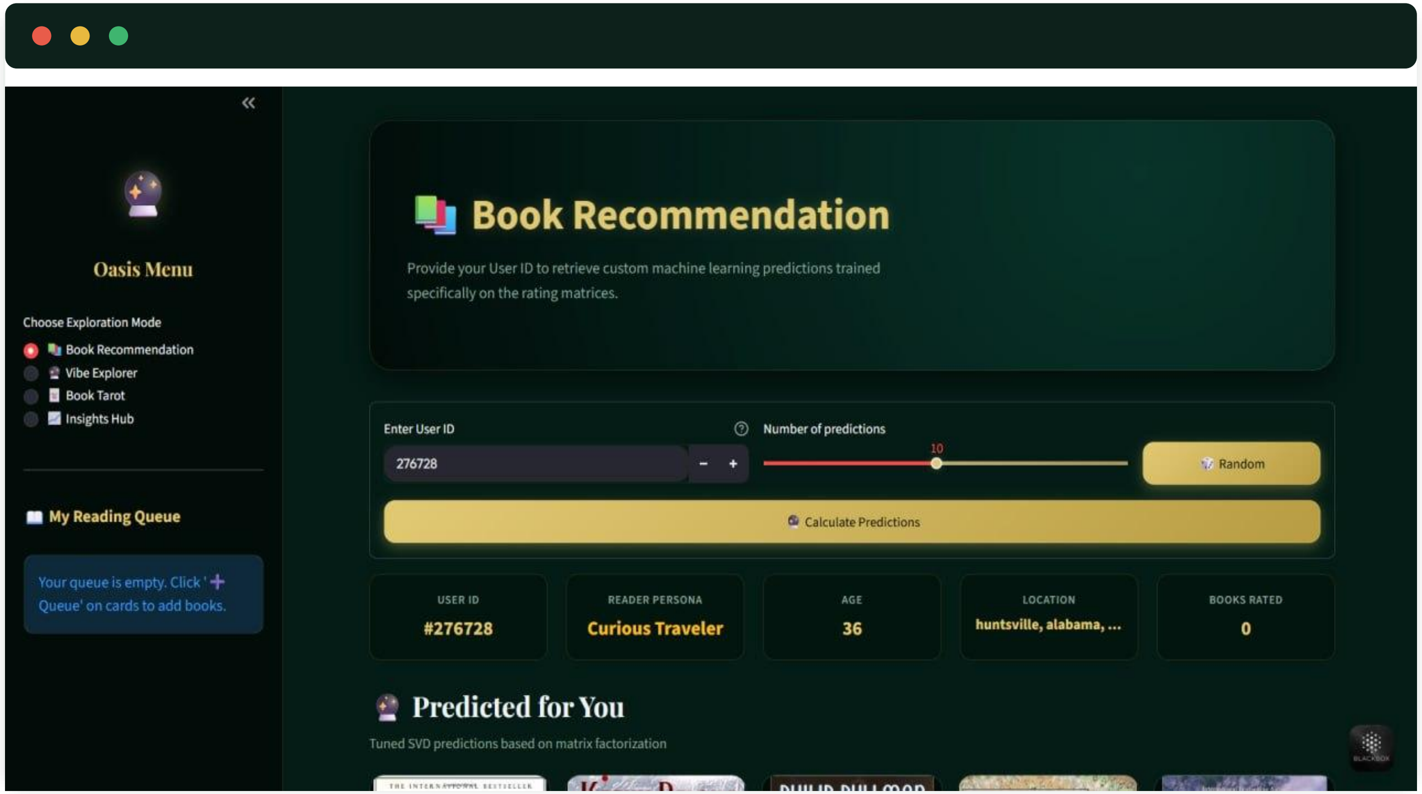
Total books, users and ratings summarised at a glance.

Illustrated Book Cards

Cover art, author, publisher and predicted rating badge per recommendation.

SEE IT IN ACTION

App Screenshot — Recommendation Dashboard



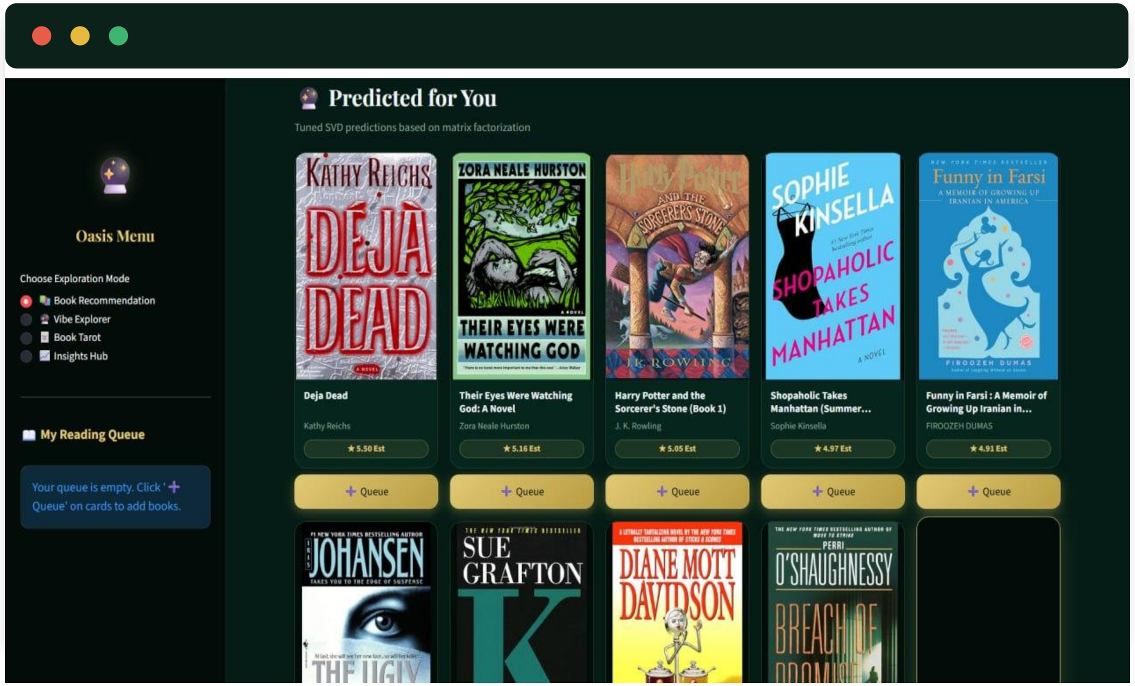
Reader-Facing Dashboard

- Reader enters a User ID and picks how many recommendations to generate.
- A live reader persona, age and location panel personalises the session.
- Predicted star rating shown alongside each cover for transparency.

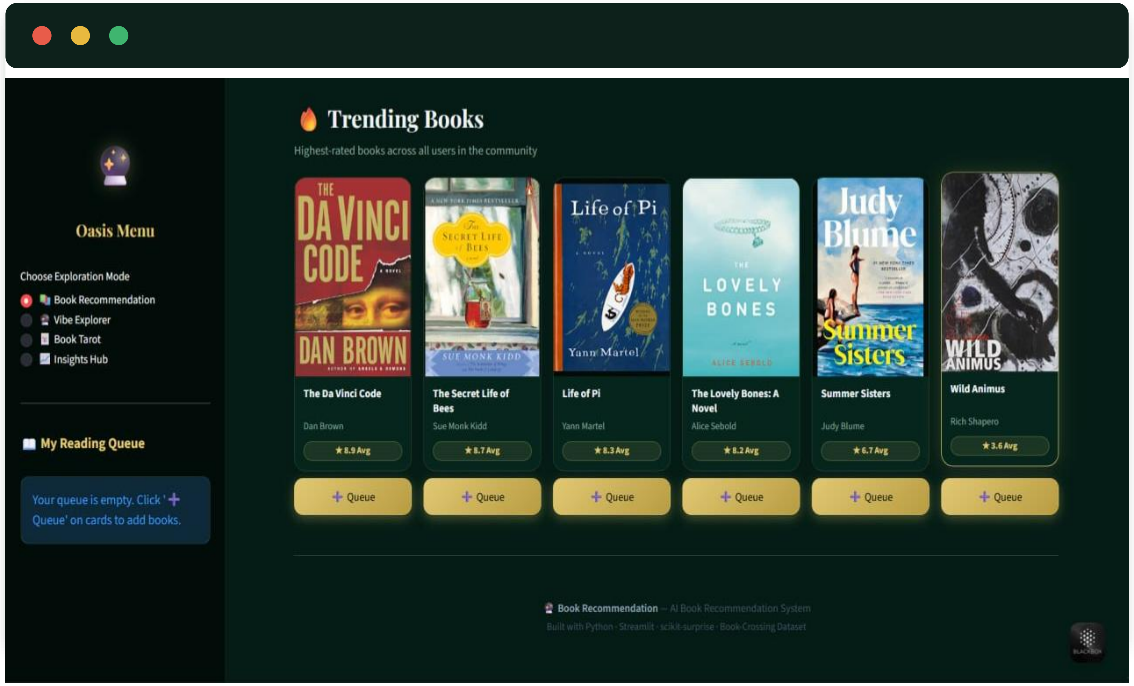
“Predicted for You” surfaces the top-ranked, previously-unrated books for that reader — ordered purely by the tuned SVD model's estimated rating, with cover art and predicted score pulled live from the pickled catalogue.

SEE IT IN ACTION

Predicted Picks & Community Trending



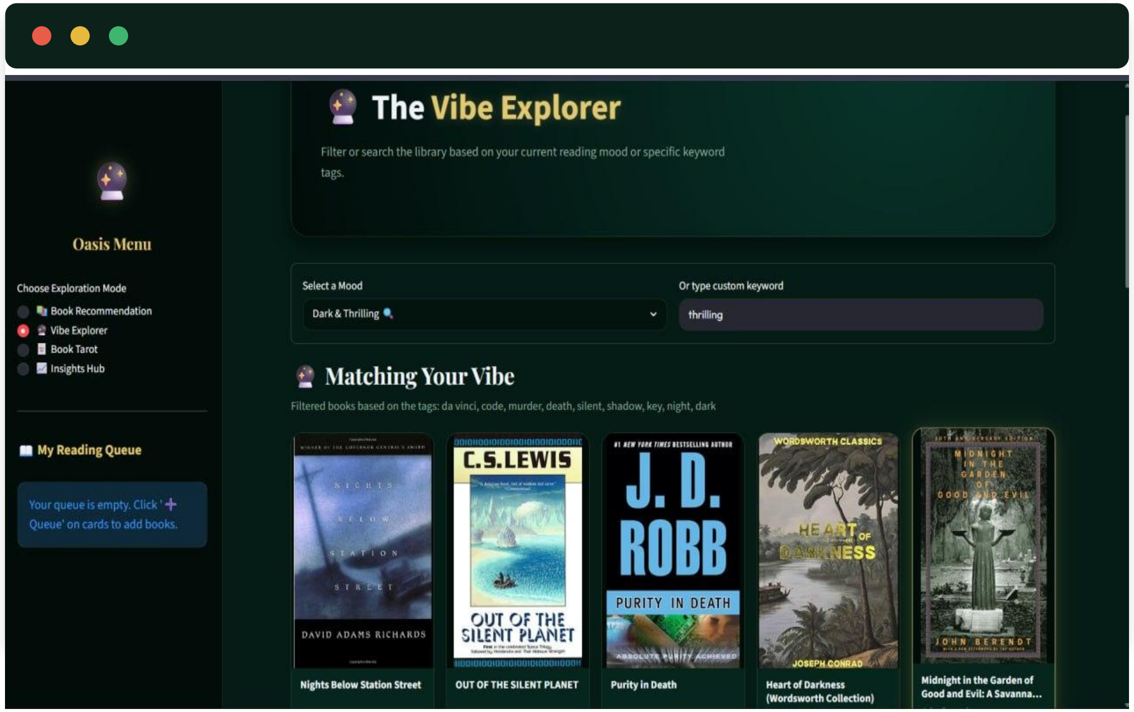
Predicted for You



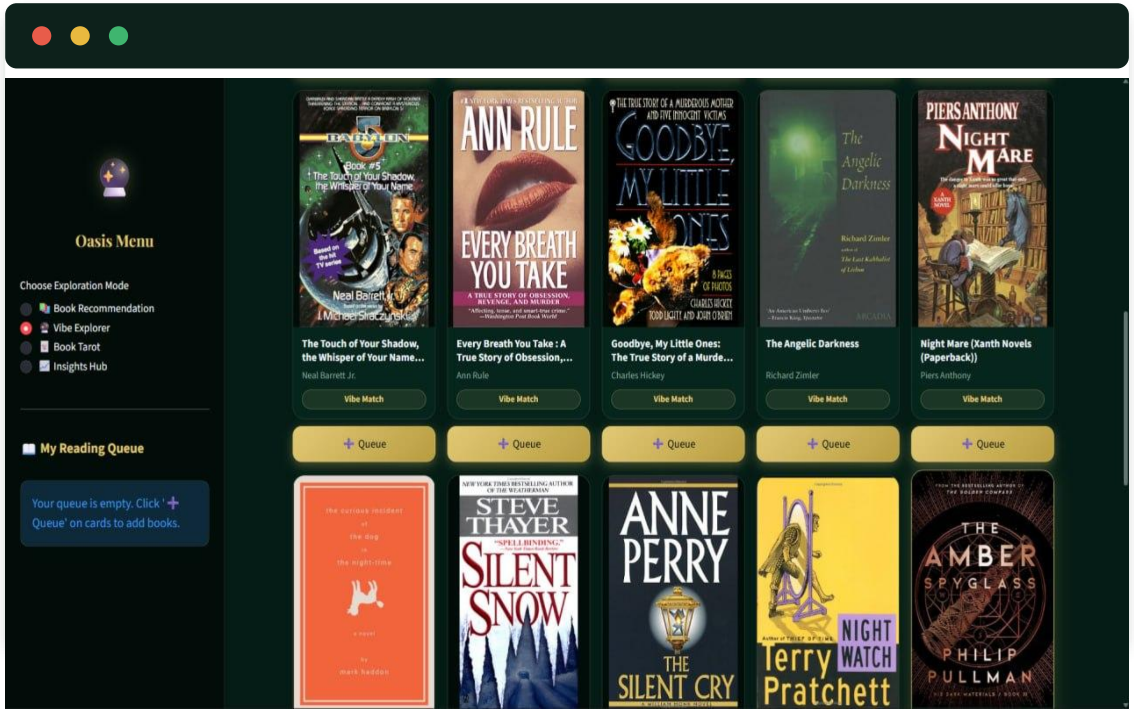
Trending Across the Community

Beyond personalised SVD predictions, the app surfaces community-wide “Trending Books” ranked by average rating — giving new or lightly-rated users a strong cold-start entry point while the personalised model has little history to work with.

The Vibe Explorer, In Depth



Vibe Explorer



More Matches

Mood-Based Search

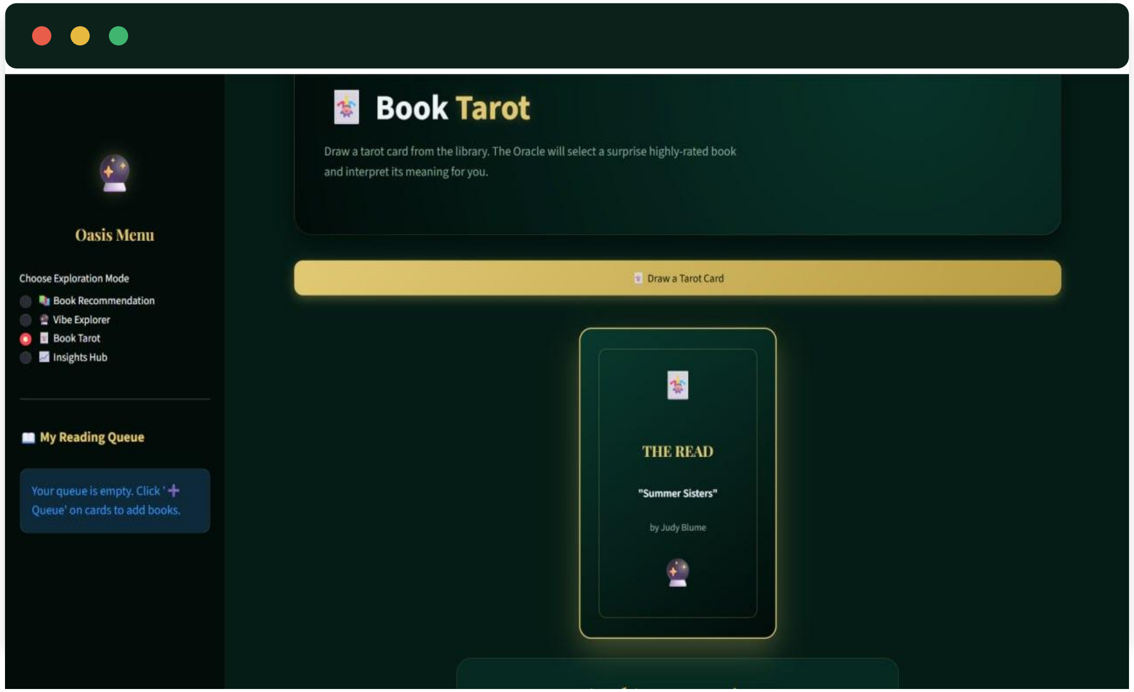
Readers filter by vibe ("Dark & Thrilling") or free-text keyword instead of a User ID.

Keep Scrolling for More

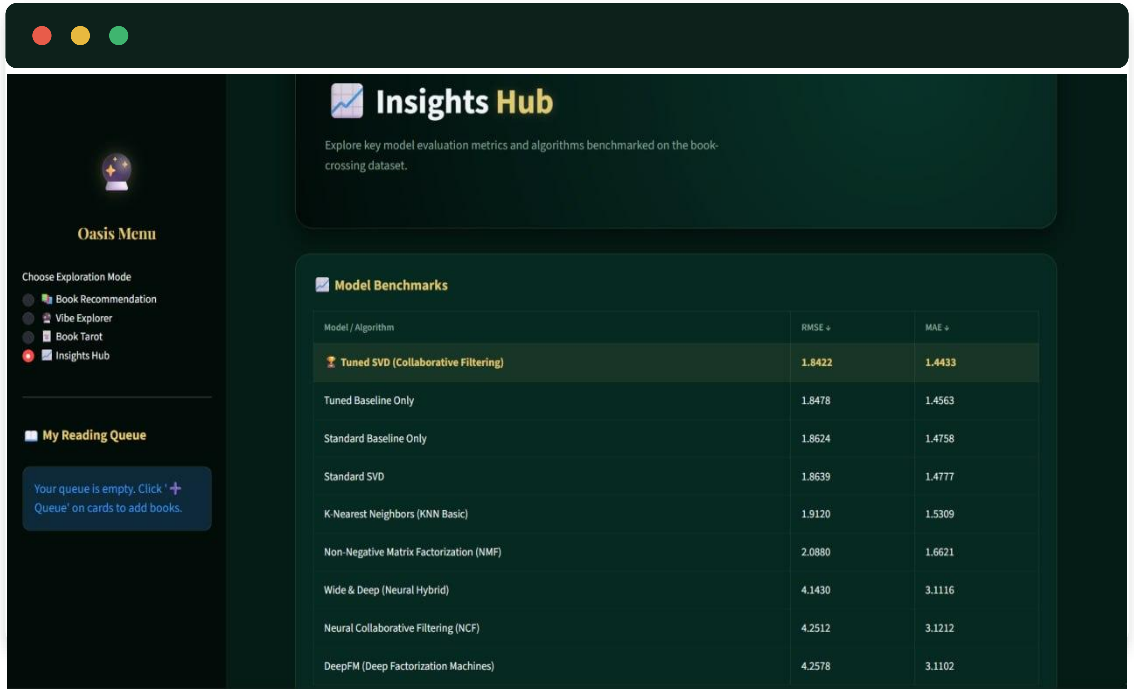
Results grid keeps loading mood-matched titles with one-tap " Queue" buttons for readers to save picks.

BEYOND THE CORE MODEL

Book Tarot & Insights Hub



Book Tarot



Insights Hub – Model Benchmarks

A third in-app mode, Book Tarot, turns discovery into a ritual — readers draw a card and the Oracle surfaces a surprise, highly-rated title with a short generated interpretation of what it means for them. Insights Hub sits alongside it as the transparent, data-driven counterpart, exposing the same nine-model benchmark table built earlier in this deck, with Tuned SVD (1.8422 / 1.4433) highlighted as the winner. Together the two modes bookend the app's personality: one leans into serendipity and mood, the other into rigor and proof.

WHY IT MATTERS

Business Value & Use Cases

CRITICAL

Longer Reading Sessions

Personalized picks reduce browsing fatigue and encourage readers to keep exploring the catalogue.

HIGH

Higher Catalogue Reach

Surfacing long-tail authors (100K+) helps mid-list and lesser-known books get discovered.

HIGH

Cross-Sell & Bundling

Predicted-rating scores can prioritise which titles to bundle, promote or discount.

MEDIUM

New-User Onboarding

Trending Books and Book Tarot give cold-start readers something compelling on day one.

MEDIUM

Retention Signal

Repeated high-confidence predictions per user can flag loyal, high-value readers.

ONGOING

Continuous Improvement

Every new explicit rating retrains and sharpens future recommendations for that reader.

Challenges Faced During the Project

Extreme Sparsity (99.99%)

Challenge: 10,254 users × 20,032 rated books leaves the matrix almost entirely empty.

Solution: Chose SVD matrix factorization over neighbourhood-based KNN, which needs direct user overlap to work well.

Explicit vs Implicit Noise

Challenge: 62% of interactions carry no genuine star rating (Book-Rating = 0).

Solution: Trained and evaluated only on the Explicit subset (Type = Explicit) to keep the target signal clean.

Cold-Start Users & Books

Challenge: SVD cannot score a user or book it never saw during training.

Solution: “Trending Books” and Book Tarot provide non-personalised fallbacks for brand-new readers.

Large Candidate Space

Challenge: 266,726 books make scoring every unrated title per user potentially slow.

Solution: Candidate books are limited to the unrated pool per user before scoring, keeping inference responsive.

Streamlit Reload Cost

Challenge: Reloading three DataFrames and a model on every interaction would be slow.

Solution: @st.cache_resource / @st.cache_data cache the pickled model and data across sessions.

Duplicate Book Records

Challenge: The same ISBN can appear more than once across merged sources.

Solution: drop_duplicates(subset="ISBN") applied both pre-scoring and post-merge to keep the list clean.

FUTURE ENHANCEMENTS

Where This Goes Next

MODEL QUALITY

Hybrid Recommender

Blend content features (Author_Popularity, Book_Age) with collaborative signal for richer scoring.

COLD START

Content-Based Fallback

Recommend by genre/author similarity for brand-new users and books absent from training.

ADVANCED ML

Neural Collaborative Filtering

Explore deep matrix factorization to capture non-linear user-item interactions SVD may miss.

FEEDBACK LOOP

Real-Time Retraining

Capture new in-app ratings and periodically retrain the SVD model on fresh data.

EVALUATION

A/B Testing Framework

Compare recommendation strategies (SVD vs hybrid) on real engagement metrics.

ACCESSIBILITY

Mobile Reading Companion

Package the Streamlit app as a mobile-responsive experience for on-the-go discovery.

Thank You!

AI Book Recommendation System

Book-Crossing Dataset | Tuned SVD | Streamlit